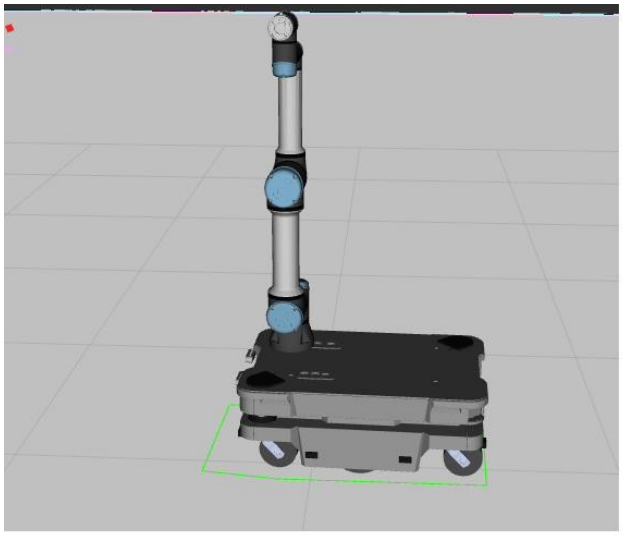
MOBILE MANIPULATOR NAVIGATION

Integrating a UR5e Arm on a MiR250 Base in Simulation



Design, Integration, and Validation of a ROS 2 Navigation Stack
for a Combined Mobile Base and Robotic Arm

Cognitive Architectures for Robotics
Project Report

ROS 2 Humble · Gazebo Classic · Nav2 · MoveIt 2

Contents

List of Figures

1. Introduction

The objective of this project was to enable a Mobile Industrial Robots MiR250 platform, carrying a Universal Robots UR5e manipulator on its top plate, to navigate safely through an indoor environment populated with everyday obstacles. A mobile manipulator of this kind is not simply two robots operating in the same space; it is a single physical system whose shape and behaviour are the product of both parts at once. The arm raises the effective height and reach of the machine well above the plane that the base's safety lasers can see, and the environment itself contains structures — tables, shelves, and other overhanging objects — whose dangerous parts sit precisely in that unseen region. Navigating such a system safely requires perception and footprint reasoning that neither the base nor the arm would need on its own.

The task was framed around four concrete requirements. The navigation costmaps had to be reconfigured to reflect the true footprint of the combined system rather than that of the bare base. Eight proximity sensors had to be integrated into the navigation stack to provide a close-range safety margin. The base's forward-facing cameras had to be used as a source of three-dimensional data so that obstacles above the laser plane could be detected. Finally, the resulting behaviour had to be demonstrated by navigating through doorways and around tables while the arm was held in a range of poses.

All of the work described in this report was carried out in simulation. This was a deliberate scoping decision rather than a limitation of the approach: the software stack used in simulation is identical to the one that would run on the physical robot everywhere above the hardware interface, so the move to real hardware amounts to swapping a single layer, not redesigning the system.

1.1 Structure of the Work

The project proceeded in stages, each building on the last. The first was to understand thoroughly how each of the two standalone robots is constructed — its description, its controllers, and its simulation plugin — because no correct modification is possible without knowing where each piece of behaviour originates. The second was the merge: combining the two descriptions into a single robot that presents one coordinate tree and one control interface. The third was to build the navigation capabilities on top of that merged robot: a footprint that follows the arm, costmaps fed by the cameras and proximity sensors, and a test environment in which the whole system could be exercised. A final study compared the available local planners to choose the most suitable one for the robot.

This report follows the same progression: the merge, the navigation stack, the dynamic footprint, the camera and proximity integration, the validation, and the planner comparison.

2. Merging the Two Robots

The MiR250 and the UR5e each arrive as a self-contained robot. Each has its own model description, its own set of controllers, its own launch procedure, and its own assumption that it is the only robot in the system. The first and most fundamental engineering problem of the project was to dissolve those assumptions and produce a single robot that the rest of the software — navigation, control, visualisation — can treat as one coherent machine. Everything that follows depends on this merge being correct, so it is treated here in detail.

2.1 Why the Two Descriptions Must Become One

A robot in ROS is defined by a single model description, published once and read by everything else in the system. From that one description a single coordinate tree is built — the structure that lets any part of the software ask where one part of the robot is relative to another. The critical word is single: the conventional bring-up assumes exactly one description, and one coordinate tree, per robot.

This is why the two robots cannot simply be launched alongside one another. Doing so would create two independent descriptions and two independent coordinate trees with no relationship between them. The system would know where the base is and, separately, where the arm is in its own private frame of reference, but it would have no way to express the arm's position in terms of the base, the map, or anything the navigation stack understands. A point at the gripper could not be related to the floor the robot drives on.

Merging resolves this by authoring a single combined description that contains both robots and states how they are physically attached to one another. The outcome is one continuous structure running from the floor reference of the base, up through the body, to the very tip of the arm. Once that structure exists, the position of any part of the arm can be traced all the way down to the map frame in a single step — which is what the navigation system needs in order to plan for the whole machine, and what the footprint computation later relies upon.

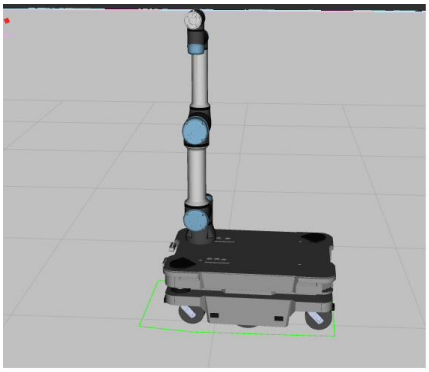


Figure 2.1 — *The combined robot in simulation, treated by the system as one machine rather than two.*

2.2 The Control Architecture Behind Both Robots

Before the two robots can share a control layer, it is worth setting out how that control layer is organised, because its structure dictates how the merge must be done. Both robots are driven by the same control framework, which is deliberately arranged in two parts: a large upper portion that is identical whether the robot is simulated or real, and a single small lower portion — the piece that actually talks to the motors — that is the only thing which changes between the two.

In the upper, shared portion, the high-level software issues goals. These are received by a central component, the controller manager, which runs the control loop at a fixed rate. Inside it sit the individual controllers: one that turns velocity commands into wheel motion for the base, one that follows joint trajectories for the arm, and one that broadcasts the state of every joint back to the rest of the system. These controllers do not touch hardware directly; they write commands into, and read feedback from, a set of shared interface buffers managed on their behalf. Only at the very bottom does the framework meet the physical world, through a single hardware component — the physics engine in simulation, a vendor driver on a real robot. Everything above this point is unchanged between the two cases.

This arrangement is the reason a stack developed in simulation transfers cleanly to hardware: the boundary between simulated and real is drawn at a single, well-defined seam, and all of the merge work happens above that seam.

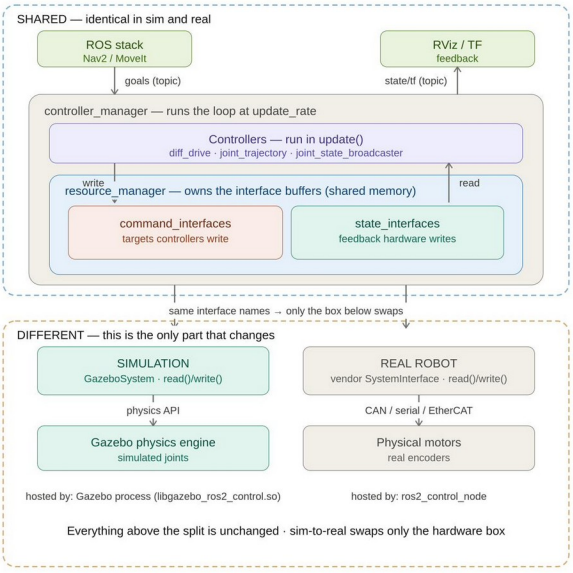


Figure 2.2 — *The control framework. Everything above the dashed line is identical in simulation and on hardware; only the bottom hardware component changes.*

The same framework can also be viewed in terms of where its configuration originates, which matters because the merge edits land in different files for different reasons. The robot description carries a block that defines what interfaces each joint exposes and names the hardware component to use; a separate configuration file lists the controllers and their parameters; and the launch procedure activates the controllers once everything is in place. Recognising this division — the description defines the interfaces, the configuration tunes the controllers, the launch procedure activates them — is what allows each merge change to be placed in the correct location rather than discovered by trial and error.

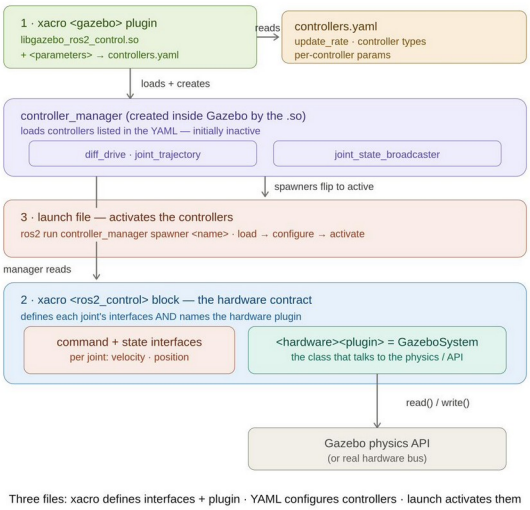


Figure 2.3 — *The three sources that together declare and activate the controllers.*

2.3 A Single, Unified Control Layer

The decisive property of this architecture, for the purposes of the merge, is that there is exactly one controller manager per simulation instance. This rules out the naive approach of running each robot's control stack independently, because two controller managers competing inside one simulation would conflict over the shared control loop. The merge therefore consolidates everything into one control layer.

Concretely, the combined robot uses one simulation plugin, which creates one controller manager, which loads a single configuration listing all the controllers from both robots together: the base's drive controller, the arm's trajectory controller, and the shared joint-state broadcaster. These are activated against that one manager, so the wheels and the arm are governed by a single coherent control layer running on the same loop, rather than by two separate stacks that would have to be coordinated externally.

It is also worth recording what this consolidation implies for an eventual move to real hardware, since the two questions — how to merge, and how to go from simulation to reality — touch the same parts of the stack. Merging collapses the two robots' control descriptions, controller configurations, and hardware components into one of each. The move to hardware, by contrast, leaves almost all of that untouched and changes only the single hardware component at the bottom. The two transformations are largely independent, which is what makes it reasonable to develop the whole system in simulation and migrate it afterwards.

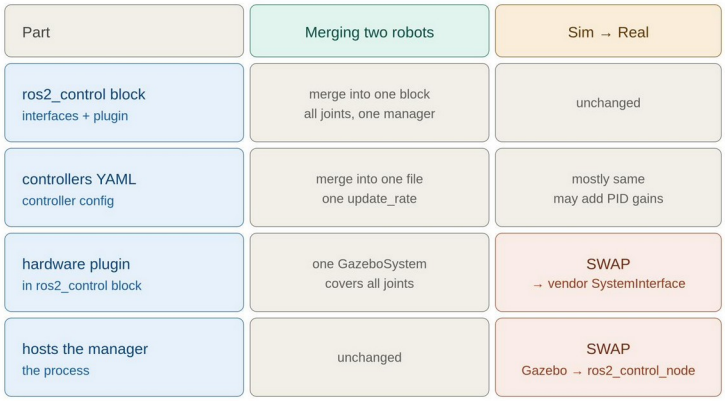


Figure 2.4 — *What each part of the control stack requires for the merge, and separately for an eventual move to hardware.*

2.4 The Combined Coordinate Tree

With a single description and a single control layer in place, the combined robot presents one coordinate tree that every other component reads. It is important to be precise about how that tree is produced, because it is not the work of any one node — it is assembled cooperatively from three contributions, each responsible for a different stretch of the chain.

The first contribution comes from localisation, which establishes where the robot sits within the map by relating the map frame to the robot's odometry frame. The second comes from the drive controller, which reports how far the base has travelled and so relates the odometry frame to the robot's ground reference. The third, and by far the largest, comes from the component that turns the static model and the live joint angles into transforms for every link: from the ground reference outward to the wheels, up through the body, along the entire arm, and out to each sensor. This third contribution is where the merge pays off, because a single such publisher, reading the single merged description, now owns the transforms for both the base and the arm together. There is exactly one authority for this part of the tree, so there is no possibility of competing or duplicated transforms.

Everything downstream — the navigation costmaps, localisation's own sensor matching, the arm planner, the footprint computation, and the visualiser — is purely a consumer of this assembled tree, asking it where one frame lies relative to another.

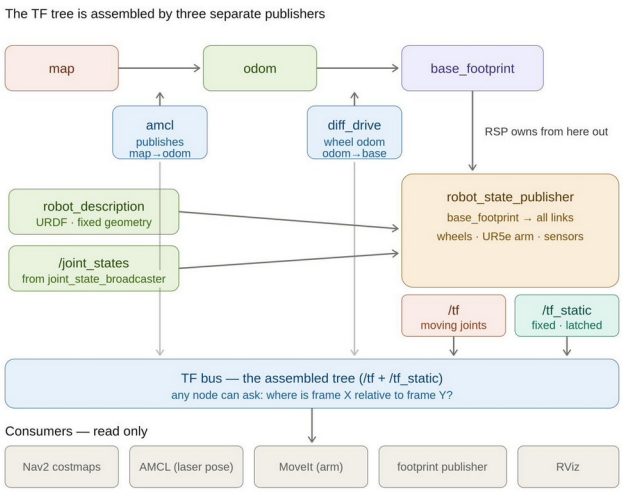


Figure 2.5 — *The combined coordinate tree is assembled by three publishers — localisation, the drive controller, and the model-state publisher — and read by many consumers.*

2.5 Why Naming Prefixes Are Indispensable

Joining the two descriptions raises a problem that is easy to overlook and difficult to diagnose once it strikes: name collisions. Within a single robot, every frame and every joint must carry a unique name, because that name is the only handle by which the coordinate tree identifies it. The two robots, however, were each authored in isolation, and both use the same generic vocabulary for their parts. Both, for instance, possess a part called the base link, along with similarly generic names for shoulders, wrists, and other elements.

If the two descriptions are combined without intervention, the merged model contains two distinct physical parts bearing the same name. The coordinate tree cannot tolerate this. Each frame is permitted only a single parent, and when two frames share a name the tree can no longer be assembled unambiguously. What makes this failure particularly troublesome is that it is silent: the description still loads, but the coordinate tree comes out broken or incomplete. The visible consequences — a malformed tree in the visualiser, transform lookups that fail, a navigation stack unable to localise — appear far from the actual cause, giving no obvious indication that a naming clash is to blame.

The remedy is to give each robot its own naming prefix, applied uniformly to every one of its frames and joints, so that no name can clash with the other robot’s. The base receives one prefix and the arm another, after which the formerly identical names become distinct and the tree assembles cleanly. The essential discipline is consistency: the prefix must be carried through not only the visible link and joint names but also the control description and the simulation plugin’s frame references. A prefix applied in some places but forgotten in others reproduces exactly the same silent breakage it was meant to prevent. For this reason the two robots were given deliberately different prefix styles, each chosen to remain compatible with the conventions of the tools that consume it downstream.

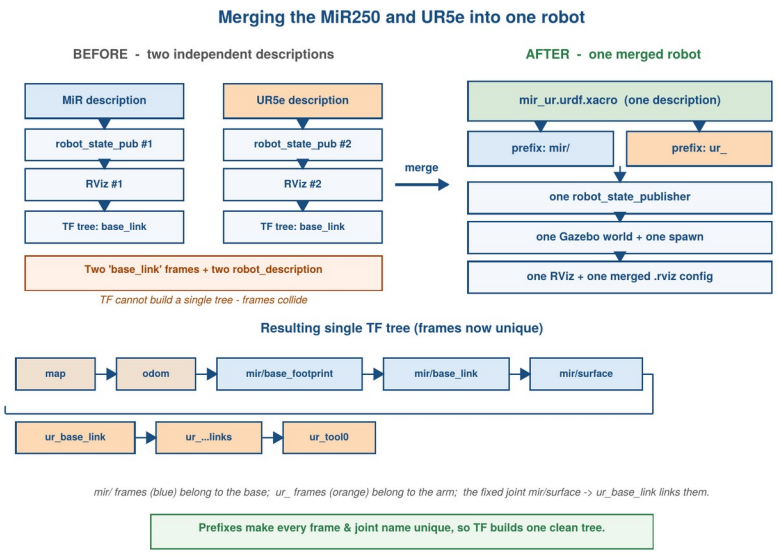


Figure 2.6 — *Before: two colliding descriptions that cannot form one tree. After: one prefixed description, one publisher, and the single resulting coordinate tree.*

The principle in one sentence: a single continuous description is what allows one coordinate tree to span both robots, and consistent naming prefixes are what allow the names within that single tree to remain unique. Omit the prefixes and the model still compiles, but the coordinate tree fails silently — among the hardest faults in the whole system to trace back to its source.

2.6 Anchoring the Arm to the Base

Unique names make a clean tree possible, but they do not by themselves connect the two robots; they merely ensure the parts can coexist. The physical connection is made by a single rigid attachment that fixes the root of the arm to a dedicated mounting point on the top of the base. This one attachment fuses two formerly separate chains into a single unbroken structure running from the base's ground reference, up through the body to the mounting point, and onward through every segment of the arm to its tip.

Because the attachment is rigid, the arm is expressed entirely relative to the moving base. Whenever the base drives or turns, the whole arm travels with it within the coordinate tree automatically. As a result, a single query can relate any point on the arm — the gripper included — to the map the robot navigates in. This is precisely the capability the later chapters build upon: the navigation stack can reason about where the arm is in the world, and the footprint computation can project the arm's current pose onto the ground to determine how much space the machine truly occupies at any moment.

Taken together, the merge yields one description, one coordinate tree, one model-state publisher, one simulation world with a single spawned robot, one controller manager carrying both robots' controllers, and one visualiser configuration. From this point onward the combined MiR250 + UR5e behaves, to every part of the software above it, as a single machine.

3. The Navigation Stack

With the two robots merged into a single machine, the project could turn to its actual purpose: making that machine navigate. All of the sensor work described in the later chapters — the dynamic footprint, the depth cameras, and the proximity sensors — exists to feed the navigation system. It is therefore worth understanding that system in its own right before describing what was added to it, because each later modification is best understood as a change to a specific, identifiable part of this stack. This chapter sets out how the navigation system is organised, how it knows where the robot is, and how it decides what counts as an obstacle.

3.1 How the Pieces Fit Together

The navigation system is not a single program but a collection of cooperating components, each with one responsibility, coordinated by a behaviour engine that decides what to do and in what order. A navigation request begins as a goal pose — a position and orientation the robot should reach. This goal is handed to the behaviour engine, which orchestrates the rest of the system by calling on a small number of specialised servers in turn.

The first of these is the planner, whose job is to compute a complete route from the robot's current position to the goal across the whole known map. This route is a global plan: it accounts for all the walls and fixed structures, but it is computed relatively infrequently and does not concern itself with the fine details of driving. The second is the controller, whose job is to follow that route. It looks only a short distance ahead, continuously generating the velocity commands that move the wheels while steering around anything that appears in the robot's immediate surroundings. When the controller cannot make progress, a set of recovery behaviours is invoked to free the robot, and a smoothing step refines the route so the motion is not unnecessarily jagged. The velocity commands the controller produces are sent to the drive controller of the base, and the robot moves.

The essential division to take away is between the planner, which reasons globally and slowly about the whole route, and the controller, which reasons locally and quickly about the next short stretch. This division reappears directly in how obstacle information is organised, and it is the key to understanding where each sensor's data belongs.

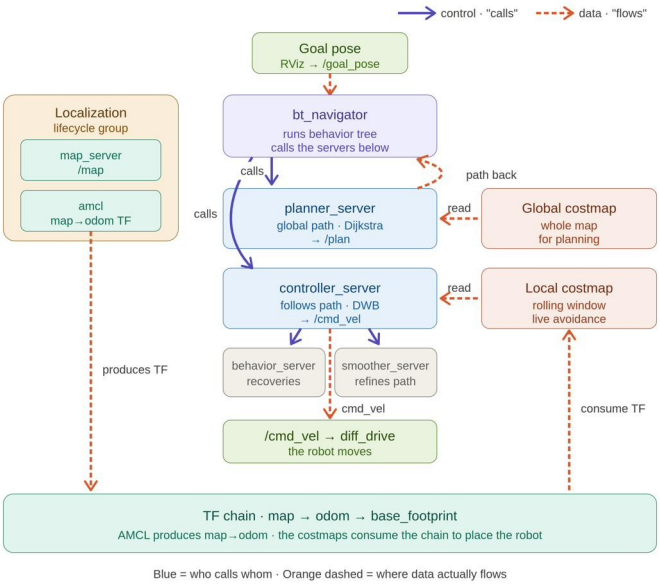


Figure 3.1 — *The navigation components. Solid arrows show which component calls which; dashed arrows show where data actually flows. The planner and controller each read from their own costmap, and both depend on the coordinate chain established by localisation.*

3.2 Knowing Where the Robot Is

None of the planning or control just described is meaningful unless the system knows where the robot currently is within the map. Establishing this is the job of localisation, and its output is what ties the entire navigation stack to the map frame the planner works in.

Wheel odometry alone is insufficient: it accurately reports how the robot has moved relative to its starting point, but it drifts over time and has no knowledge of where that starting point lies on the map. Localisation closes this gap by continuously comparing what the laser sees against the known map and correcting the robot's estimated position to make the two agree. It does this probabilistically, maintaining a population of several hundred to a couple of thousand hypotheses about where the robot might be. Each cycle these hypotheses are moved according to the odometry, scored by how well the current laser scan would match the map at that hypothesis's location, and resampled so the better-matching hypotheses survive. Over time the population converges on the robot's true pose.

Crucially, localisation does not republish the whole coordinate chain. It contributes only the single correction relating the map to the robot's odometry frame; the odometry itself supplies the link from there down to the robot's ground reference. Together these produce the complete chain that the planner and both costmaps read in order to place the robot in the world. Localisation relies solely on the laser; it makes no use of the cameras or proximity sensors added later, which serve the costmaps rather than the position estimate.

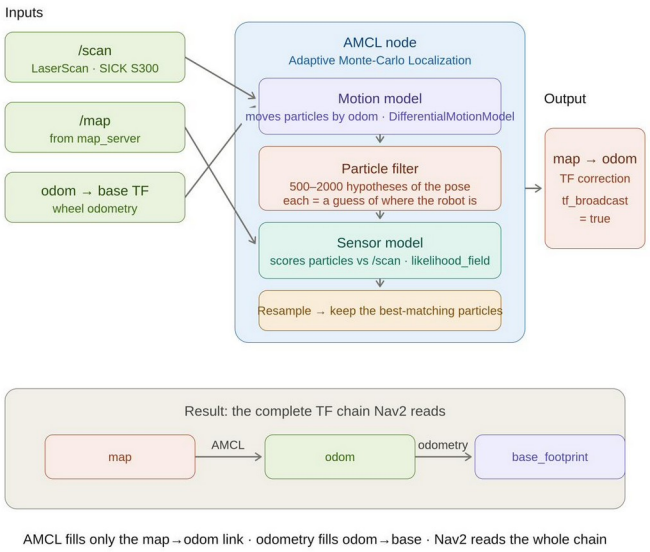


Figure 3.2 — *How localisation turns the laser, the map, and odometry into the single map-to-odometry correction, completing the coordinate chain the navigation stack reads.*

3.3 The Costmaps: How Obstacles Are Represented

The planner and the controller do not reason about raw sensor data directly. Instead, each consults a costmap — a grid covering the robot's surroundings in which every cell carries a cost value expressing how undesirable it is to occupy. High cost marks an obstacle or its immediate vicinity; low cost marks free space. The planner searches for a route that keeps cost low; the controller steers to avoid high-cost cells in real time. Understanding the costmap is the prerequisite for everything in the sensor chapters, because integrating a new sensor means, in practice, arranging for it to contribute cost to these grids in the right way.

A costmap is not a single monolithic map but a stack of layers, each contributing information from a different source, combined into one master grid that is what actually gets planned on. The lowest layer is the static map of the environment's permanent structure. Above it sit the layers that mark obstacles detected live by the sensors. At the very top, an inflation layer spreads a graduated safety margin outward from every obstacle, so the planner keeps a comfortable clearance rather than grazing walls. The combined result of all the layers is the single grid of costs that the planner and controller read.

This layered design is what makes the later sensor integration tractable: each new sensor is added as, or into, a specific layer without disturbing the others. It also surfaces an important compatibility point. The layer used for the depth cameras accepts three-dimensional point data and turns it into cost — the mechanism by which overhead obstacles such as tabletops come to be avoided — but that same layer does not accept the simple distance readings the proximity sensors produce, which therefore require a separate dedicated layer. Both integrations are described in their own chapters; what matters here is that the costmap's layered structure is the framework into which they fit.

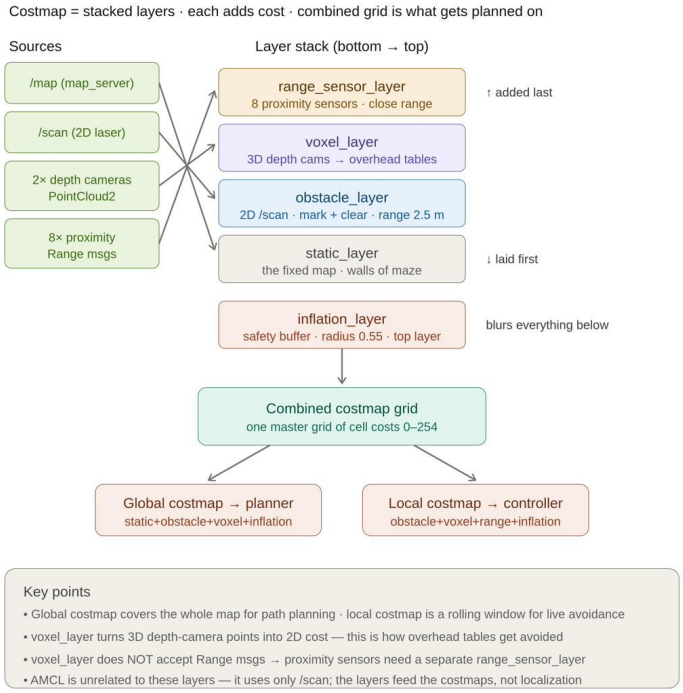


Figure 3.3 — *The costmap as a stack of layers. The static map, the laser- and camera-based obstacle layers, the dedicated range-sensor layer, and the inflation layer combine into one grid, which feeds both the global costmap used by the planner and the local costmap used by the controller.*

The two costmaps differ in scope along the global-versus-local division introduced earlier. The global costmap spans the whole map and is what the planner uses to lay out a complete route; it is built to be comprehensive. The local costmap is a small window that travels with the robot, continuously scrolling so that obstacles which fall behind are forgotten as new ones appear; it is what the controller uses for immediate avoidance. This distinction later determines where the depth-camera data is allowed to contribute, for reasons of both correctness and computational cost taken up in the camera chapter.

In summary: the planner reasons globally over the whole map, the controller reasons locally over a moving window, localisation supplies the coordinate chain that ties both to the map, and the costmaps are the layered grids through which every sensor ultimately expresses itself. Every modification in the following chapters is, at bottom, a change to one of these well-defined parts.

4. The Dynamic Footprint

The first of the four task requirements was to make the navigation costmaps reflect the true footprint of the combined system. The footprint is the outline of the robot projected onto the ground; the navigation stack uses it to judge whether a given position would put the robot in collision, and to keep a safe clearance from obstacles. For an ordinary mobile base this outline is a fixed rectangle, because the base never changes shape. The mobile manipulator is different: the moment an arm is mounted on top, the outline the machine occupies depends on what the arm is doing.

4.1 Why a Fixed Footprint Is Not Enough

When the arm is folded compactly over the base, the system's outline is close to the bare rectangle of the base itself. But when the arm reaches out — forward, or to the side — its links extend beyond the edge of the base, and the true outline grows accordingly. A footprint that is fixed cannot represent both situations, and both ways of choosing a fixed one are unsatisfactory.

If the fixed footprint is set to the bare base, then whenever the arm extends, the parts reaching past the base edge are invisible to the navigation system: the robot would believe it fits through a gap that the extended arm would actually collide with. This is unsafe. If instead the fixed footprint is enlarged to always enclose the fully extended arm, it is safe but needlessly cautious: even with the arm folded away the robot would behave as though it were as wide as its maximum reach, refusing to enter perfectly passable spaces. Neither a footprint that is always small nor one that is always large captures a machine whose real extent changes from moment to moment. What is required is a footprint that tracks the arm's actual configuration in real time.

4.2 The Footprint Projector

To provide this, a dedicated node was written that recomputes the footprint continuously and supplies it to the navigation system. The principle is to take the parts of the robot that matter — the base and every segment of the arm — project them down onto the ground plane as they currently sit, and wrap the whole projection in a single outline. As the arm moves, that outline expands and contracts to match.

The recomputation is triggered by the stream of joint-state updates: each time the arm's joints report new angles, the arm has moved, and the footprint is due to be recalculated. The node then asks the coordinate tree for the current position of each arm segment relative to the base's ground reference, keeping only the horizontal components — this is what it means to project the arm onto the floor. Because a real link is not an infinitely thin line but a solid object with thickness, each segment is represented not by a single line but by a series of points sampled along its length, each carrying a small circle of the link's actual radius. The four corners of the base are added to this same collection of points. Finally, the entire collection is wrapped in its convex hull — the smallest convex outline that contains every point — and that outline is published as the footprint. A steady timer republishes the most recent outline so the navigation system always has a current footprint to work with, even between arm movements.

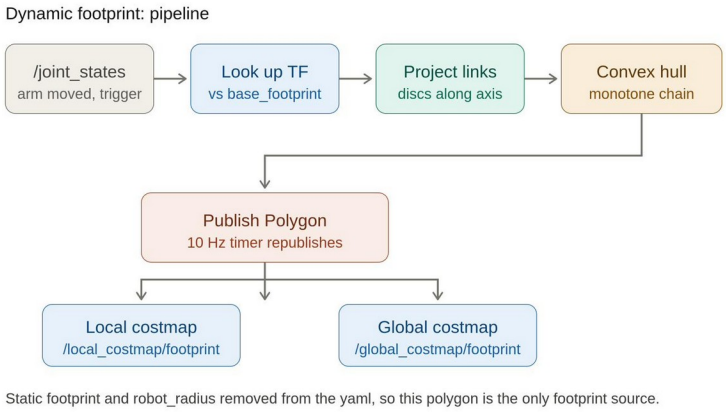


Figure 4.1 — *The footprint pipeline. A joint-state update triggers a coordinate lookup of each arm segment, the segments are projected onto the ground, the whole set of points is wrapped in a convex hull, and the resulting polygon is published to the costmaps.*

4.3 Why a Convex Hull

The choice to wrap the projected points in their convex hull, rather than tracing their exact outline, deserves explanation because it is deliberate and consequential. The navigation system requires the footprint to be a single, simple polygon. The convex hull satisfies this by construction: it is always a single convex shape, it always contains every one of the input points, and it therefore never underestimates how much space the robot occupies. Underestimating the footprint would be dangerous — it would invite collisions — whereas the convex hull errs, if at all, on the side of enclosing slightly more space than strictly necessary, which is the safe direction.

It is worth being precise about what the hull does and does not preserve, because the supporting figure can be read too literally. The circles sampled along each link are inputs to the computation, a means of giving each link its proper thickness; they are not themselves the output. Once the hull is computed, those interior circles are gone, and what remains is only the single outer boundary that encloses them all. The published footprint is one convex polygon with no internal structure. The hull itself is computed by a standard, efficient method that sorts the points and sweeps through them, building the outline while discarding any point that would create a concave indentation — fast enough to run on every cycle over the few hundred points the projection produces.

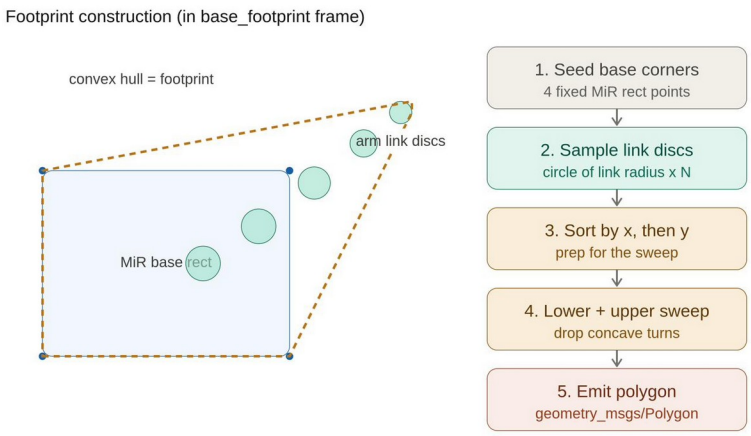


Figure 4.2 — *Footprint construction. The base corners and the per-link sample circles (shown as discs only to indicate the inputs) are wrapped by the convex hull on the left; the steps on the right summarise the method. The discs are inputs only — the published footprint is the single outer polygon.*

4.4 Integration Into the Navigation Stack

The navigation costmaps can obtain their footprint in one of two ways: from a fixed value written in their configuration, or from a live stream supplied on a dedicated channel. To make the dynamic footprint take effect, two things were arranged. First, the node publishes its continuously updated outline on the channels that both the local and global costmaps watch for footprint updates, expressing it in the base's ground-reference frame so that it moves and rotates with the robot automatically. Second, and equally important, the fixed footprint and radius values were removed from the navigation configuration. Had a fixed value been left in place, it would have taken precedence and silently overridden the live stream, and the dynamic behaviour would never have appeared. With no fixed value present, the costmaps fall back to the live channel, and the projected polygon becomes the sole source of the robot's footprint.

The result is a footprint that is honest at every instant: close to the bare base when the arm is folded, so the robot can use narrow gaps, and grown to include the arm's reach when it extends, so the planner keeps the whole machine clear of obstacles. The navigation system always reasons about the space the robot genuinely occupies, rather than a fixed approximation that is either too optimistic or too conservative.

5. Depth Cameras for Overhead Detection

The third task requirement was to use the robot's forward-facing cameras as a source of three-dimensional information so that obstacles the lasers cannot see would still be avoided. This chapter explains why those obstacles are invisible to the lasers, how the cameras were placed and made to produce usable data, and how that data was brought into the navigation costmaps.

5.1 Why the Lasers Are Not Enough

The MiR's safety lasers scan in a single horizontal plane a short distance above the floor. Within that plane they are excellent, but they are blind to anything that does not intersect it. The problematic case is an object whose base is thin but whose upper part is wide — a table being the obvious example. At the height of the laser plane the table presents only its narrow legs or central column, so the lasers report a small, easily-avoided obstacle. The wide tabletop, sitting above the laser plane, is never seen at all. A navigator relying on the lasers alone would conclude that it can pass through the apparently open space beneath the overhang, and would drive the upper body of the robot, or the arm, straight into the tabletop.

This is precisely the gap the cameras close. A depth camera perceives a volume rather than a plane: it returns the distance to whatever fills its field of view, producing a dense cloud of three-dimensional points. Those points include the parts of an obstacle that lie above the laser plane — exactly the information needed to recognise an overhang as something to avoid.

5.2 Placing the Cameras

Two cameras were mounted on the front of the robot, positioned high on the body and oriented horizontally so that each looks straight ahead. The two are parallel rather than splayed; together they cover the space directly in front of the machine, which is the direction it travels and therefore the direction in which overhead obstacles must be detected in time to react. The cameras' exact mounting position is not specified in the manufacturer's datasheet, so a sensible forward-facing placement was chosen for the simulated model; should a precise mounting be required later, the camera in the model can simply be moved to it without any other change.

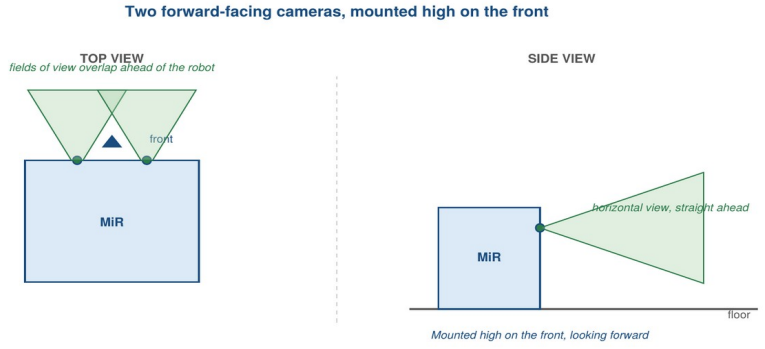


Figure 5.1 — *The two cameras sit high on the front of the robot and look straight ahead, horizontally. Their fields of view overlap ahead of the robot, covering the space directly in front — including the region above the laser plane where overhangs live.*

5.3 Making the Cameras Produce Data

A camera in the robot model is, by itself, only a placeholder; it produces nothing until a simulation sensor plugin is attached to it. The plugin was configured to operate the camera as a depth sensor, so that on every cycle it samples the simulated scene and publishes a depth image together with the corresponding three-dimensional point cloud. It is this point cloud — a live set of

points describing the surfaces in front of the robot — that the navigation system consumes. Both cameras are produced from a single reusable description so that they behave identically and differ only in where they are mounted.

5.4 Bringing the Cameras Into the Costmap

Recall that obstacles reach the planner and controller only through the costmap, and that the costmap is built from layers, each suited to a particular kind of sensor data. The camera point clouds are fed into the layer designed for three-dimensional input. This layer takes the cloud of points, considers their height, and converts the relevant ones into cost on the two-dimensional grid the navigation system plans on. This conversion from a three-dimensional cloud to two-dimensional cost is the entire mechanism by which an overhanging tabletop comes to be avoided: the points belonging to the tabletop, which sit above the laser plane, are turned into a high-cost region on the grid, and the planner routes around it just as it would around a wall.

One detail in this layer is essential to its working correctly. Because the cameras look forward, part of what they see is the floor itself a short distance ahead. Left untreated, every floor point would be turned into an obstacle, and the robot would believe it was permanently boxed in by a wall directly in front of it. The layer is therefore configured to ignore points below a small height threshold, so the floor is filtered out while genuine obstacles standing on it are retained.

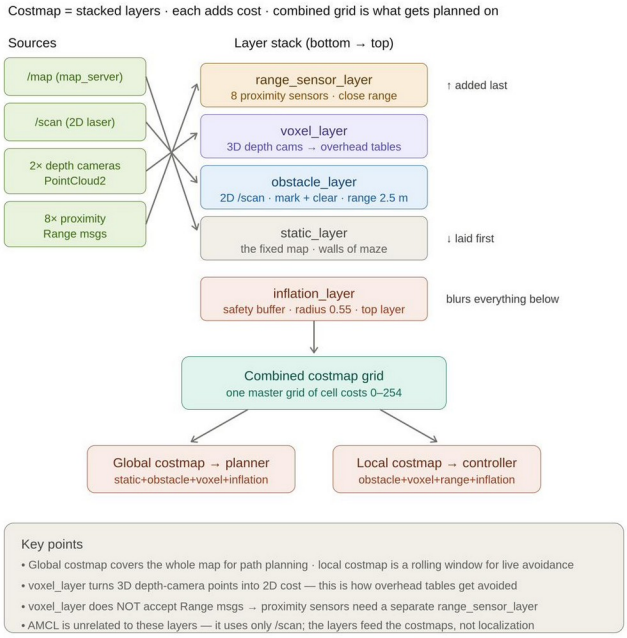


Figure 5.2 — *The cameras feed the three-dimensional (voxel) layer, which turns their points into cost — the path by which overhead obstacles become visible to the planner. The proximity sensors, discussed next, require a separate layer.*

5.5 Local Costmap Only, and Why

The camera data was added to the local costmap — the small window that travels with the robot — but deliberately not to the global costmap that spans the whole map. The reason is computational. Maintaining a full three-dimensional representation is comparatively expensive, and doing so over the small, constantly-recycled local window is entirely manageable, whereas doing the same over the entire static-sized global map proved too costly to sustain. Confining the camera contribution to the local costmap keeps the system responsive while still achieving the goal: the table only

needs to be detected once the robot is near enough to manoeuvre around it, and by that point it lies well within the local window, where the controller handles the avoidance in real time.

If global awareness of such obstacles were genuinely required — so that the long-range planner, too, would route around them from a distance — the appropriate solution would not be to extend the expensive three-dimensional layer across the global map. It would instead be to flatten the camera data into a simple two-dimensional obstacle layer for the global costmap. The global planner only ever produces a two-dimensional route, so it has no need of the full three-dimensional structure; a flattened projection would give it the obstacle's footprint at a fraction of the cost. This was noted as the natural extension but was not required for the present work.

In short: the cameras feed the three-dimensional layer because that is what turns their points into the cost that makes overhangs avoidable; they are confined to the local costmap because a three-dimensional layer over the whole global map is too expensive; and if global coverage were ever needed, a flattened two-dimensional projection into the global obstacle layer would be the economical way to provide it.

6. Proximity Sensors

The second task requirement was to integrate eight proximity sensors into the navigation. These are short-range sensors distributed around the chassis, intended as a close-in safety margin that complements the medium-range cameras and the planar lasers. This chapter describes how they were placed and modelled, how they differ from their real-robot counterparts, and why they required a costmap layer of their own.

6.1 Placement and Modelling

Eight sensors were arranged symmetrically around the perimeter of the base: two facing forward, two facing backward, and two on each side. Each was modelled as a short-range sensor producing a single distance reading, oriented horizontally so that its detection region extends outward from the chassis parallel to the floor. They were deliberately kept horizontal rather than tilted toward the ground, and their detection region was kept narrow so that it does not strike the floor and report it as an obstacle. All eight are generated from one reusable description, differing only in their mounting position and the direction each faces, which keeps them uniform and easy to adjust together.

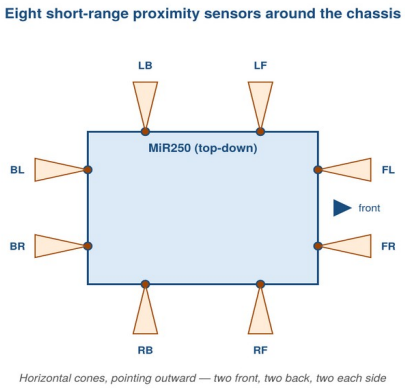


Figure 6.1 — *The eight sensors arranged around the chassis, each looking outward, horizontally — two front, two back, and two on each side — giving close-range coverage all around the robot.*

6.2 How the Model Differs From the Real Robot

It is worth being explicit about a difference between this simulated model and the proximity sensors on the physical MiR, because the two serve different purposes. On the real robot the proximity sensors are angled down toward the floor and are wired into the machine's safety system: when something comes too close, they cut the motors and bring the robot to a stop, acting as a hardware-level emergency measure independent of the navigation software.

Figure 6.2 — *The proximity sensors on the physical MiR, set into the angled corner recesses and directed toward the floor. On the real robot these trigger a hardware motor cut-off when an obstacle is close — a behaviour the simulated model deliberately does not reproduce.*

The model built here does not reproduce that motor-cut-off behaviour. Instead, the sensors were modelled to serve normal navigation: their readings are fed into the costmap so that the planner and controller treat a nearby object as an obstacle to be steered around, in the same way they treat any other obstacle. This is a deliberate choice of scope — the sensors contribute to ordinary obstacle avoidance rather than to an emergency stop — and it is the reason they are oriented for forward sensing rather than for watching the ground directly beneath the robot's edge.

6.3 Why a Separate Costmap Layer

The proximity sensors could not be added to the same layer as the cameras. That layer accepts only the richer sensor types — planar laser scans and three-dimensional point clouds — and it rejects the simple single-distance readings that the proximity sensors produce. Attempting to feed the proximity readings into it fails outright.

The correct home for these readings is a dedicated layer built specifically for distance sensors. This layer understands a single-distance reading: it interprets each one as a short cone projecting outward from the sensor, marks an obstacle where the reading indicates something is present, and clears the cone again when the reading shows the way is open. With the eight sensors registered in this layer, their readings contribute to the same combined costmap as the lasers and cameras, and the navigation system reacts to a close obstacle on any side of the robot. Apart from living in its own layer, the integration is otherwise ordinary: the layer takes its place in the costmap's stack alongside the others, and the planner and controller consume the combined result without needing to know which layer a given piece of cost came from.

In short: the proximity sensors are modelled as horizontal, outward-facing short-range sensors around the chassis, serving normal obstacle avoidance rather than the hardware motor cut-off used on the real robot. They live in a dedicated distance-sensor layer because the camera and laser layer does not accept their simple distance readings; from the planner's point of view they then behave like any other obstacle source.

7. Validation

With the system complete, the final task was to demonstrate that it behaves as intended. Validation was carried out against the specific capabilities the project set out to deliver: that the footprint follows the arm, that the robot respects doorways differently depending on the arm's pose, and that the cameras allow overhead obstacles to be avoided where the lasers alone would fail. This chapter describes the test environment and then presents each demonstration in turn.

7.1 A Purpose-Built Test Environment

The standard demonstration world was not suited to these tests, so a new environment was created specifically to exercise the features of interest. It consists of a walled area divided into rooms by interior partitions, with a narrow doorway whose width is comparable to the robot's own, and two free-standing tables placed in the open floor. The tables are the crucial element: each has a narrow base at floor level but a much wider top above it, so that at the height of the safety lasers only the slender base is visible while the broad tabletop overhangs the apparently empty space around it. This is exactly the configuration that defeats laser-only navigation and that the cameras are meant to handle.

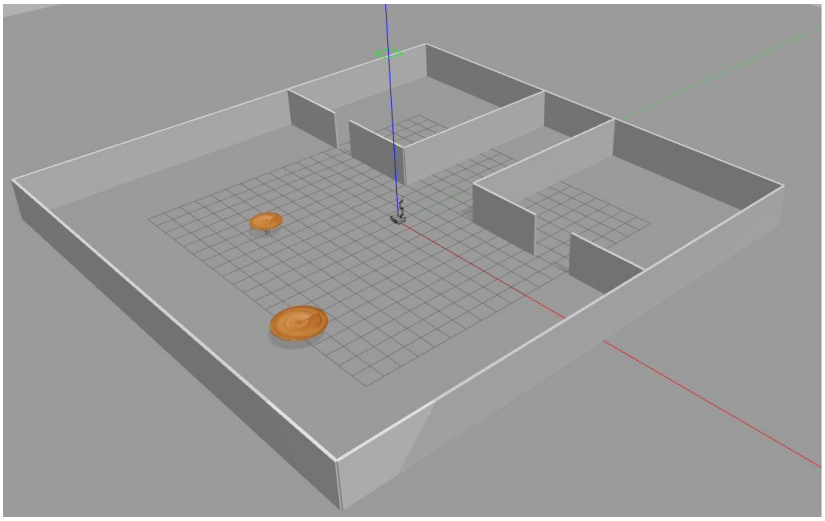


Figure 7.1 — *The purpose-built test environment: interior partitions forming rooms with a tight doorway, and two tables whose wide tops overhang their narrow bases — visible to the cameras but not to the lasers.*

Before this environment could be navigated, a map of it had to be produced. Mapping is performed once, ahead of time: the robot is driven manually through every room and hallway while the mapping system matches successive laser scans and grows an occupancy grid of the space. Once coverage is complete, the finished grid is saved to disk as a reusable map. During later navigation runs this saved map is simply loaded and reused; the live mapping process is not repeated. The tabletops do not appear in this map, precisely because the mapping lasers never see them — which is what makes the later camera demonstration meaningful.

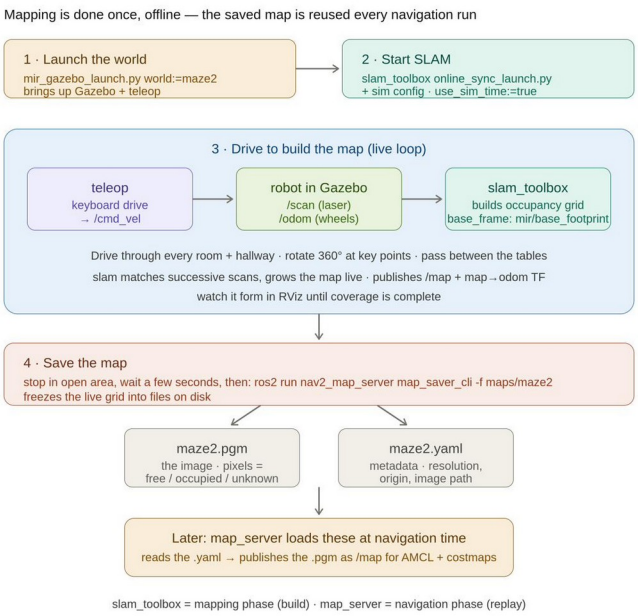


Figure 7.2 — *The map is built once by driving the robot around while the mapping system assembles an occupancy grid, which is then saved and reused for every subsequent navigation run.*

7.2 The Footprint Follows the Arm

The first demonstration confirms that the dynamic footprint behaves as designed. With the arm extended out to the side of the base, the published footprint is seen to stretch outward to enclose the arm, rather than remaining the bare rectangle of the base. The outline is no longer a tidy box around the chassis but a larger polygon that reaches out to wrap the projecting arm, exactly as the projection-and-hull process intends. This is the visible confirmation that the navigation system is reasoning about the true extent of the machine at each instant.

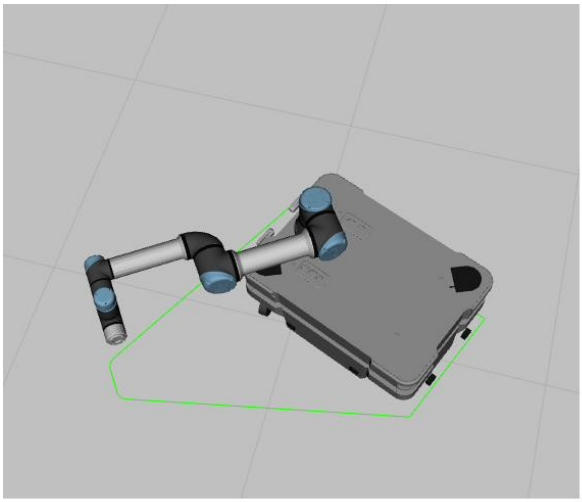


Figure 7.3 — *The dynamic footprint in action: with the arm extended to the side, the published outline grows to enclose the arm rather than just the base.*

7.3 The Doorway Test: Arm Pose Decides Passage

The second demonstration shows the practical consequence of that dynamic footprint at the tight doorway, and it is the clearest evidence that the footprint is genuinely driving the navigation rather than merely being displayed. The same goal was set on the far side of the narrow doorway, and the robot was asked to reach it under two different arm configurations.

With the arm extended, the enlarged footprint does not fit through the doorway. The navigation system, correctly reasoning that the machine in its current shape cannot pass, is unable to find a valid way through, and the robot does not make it past the opening. When the arm is then retracted, the footprint shrinks back toward the base outline, the same doorway now offers enough clearance, and the robot passes through to reach the goal. The only thing that changed between the two runs was the arm's pose; the footprint changed with it, and the navigation outcome changed accordingly. This is the dynamic footprint demonstrating its entire purpose.

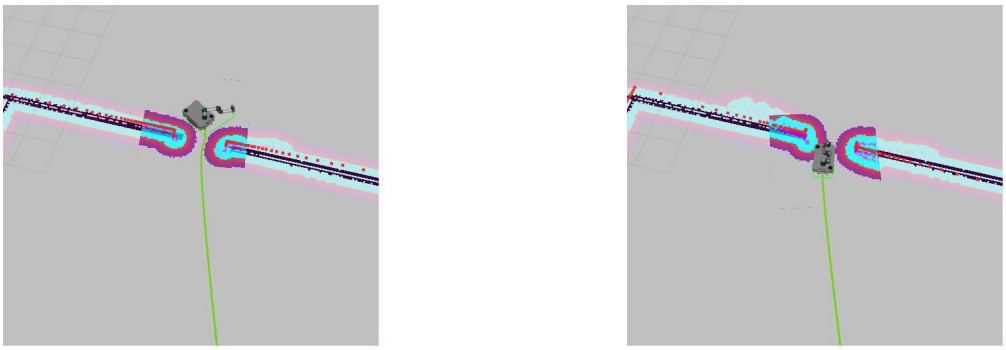


Figure 7.4 — *Left: with the arm extended, the enlarged footprint cannot fit through the narrow doorway and the robot cannot reach the goal. Right: with the arm retracted, the smaller footprint clears the same doorway and the robot passes through.*

7.4 The Camera Test: Avoiding an Overhead Table

The third demonstration validates the camera integration, and with it the central perception goal of the project. A goal was placed on the far side of one of the tables, so that the most direct route would pass straight through the space the tabletop overhangs.

When the goal is first issued, the global planner — which works only from the pre-built map, and that map contains no record of the tabletop — lays out a direct route that passes right through the table's location. At this stage the robot has no knowledge of the obstacle: from the map's point of view the path is clear.

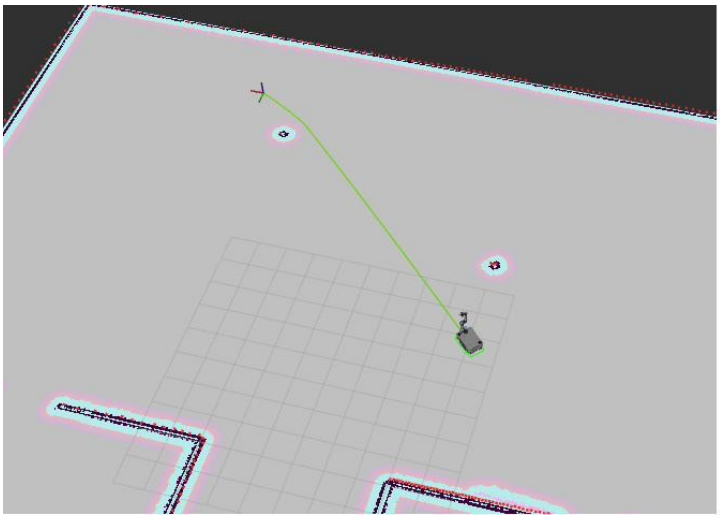


Figure 7.5 — *The initially planned route runs straight through the table's position, because the pre-built map does not contain the overhanging tabletop.*

As the robot advances and the table comes within range of the forward cameras, the situation changes. The cameras perceive the wide tabletop — the part that sits above the laser plane — and that detection is turned into cost in the local costmap, marking the region as occupied. Confronted with this newly-discovered obstacle, the navigation system revises its route, bending the path around the table rather than through it, and the robot proceeds safely to the goal.

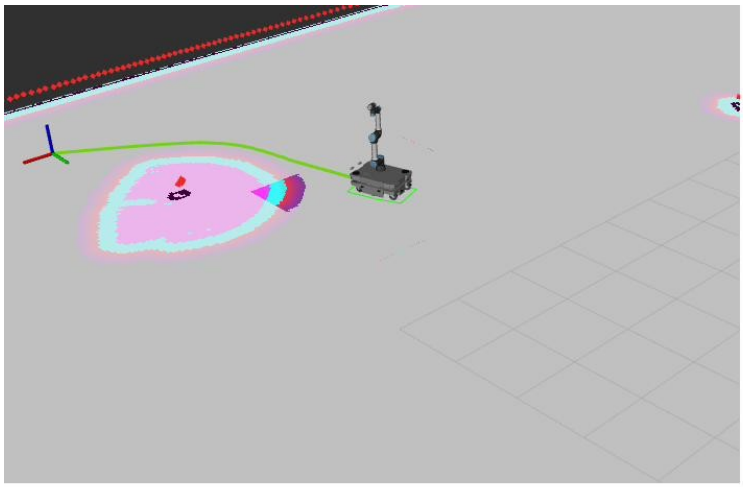


Figure 7.6 — *As the robot approaches, the cameras detect the overhanging tabletop (the marked region) and the route is corrected to curve around it instead of driving through.*

The significance of this test lies in what would happen without the cameras. Relying on the lasers alone, the robot would perceive only the table's narrow base and would conclude that the wide space around it was free; it would follow the original straight route and drive the upper body of the machine into the overhanging tabletop. The cameras are precisely what prevent this, and the corrected path is the direct evidence that the overhead-detection requirement has been met.

Across the three demonstrations the system meets its goals: the footprint expands and contracts with the arm, the doorway is passable only when the arm is retracted, and an overhead table invisible to the lasers is detected by the cameras in time to reroute around it. Together these confirm that the merged robot navigates as a single machine aware of both its

own changing shape and the obstacles its lasers cannot see.

7.5 A Note on the Proximity Sensors

The proximity sensors were integrated into the costmap as described in the previous chapter, and their readings do contribute to the combined obstacle representation. In the validation runs, however, their individual effect on the robot's behaviour was not clearly distinguishable. This is to be expected: their close-range coverage overlaps with regions already well covered by the lasers and cameras, so in these particular scenarios they rarely provided the deciding information. Their contribution is best understood as a redundant short-range safety margin that would matter most in situations — very near obstacles, or sensing gaps of the other sensors — that these specific tests did not isolate. A dedicated scenario placing an object within the proximity band but outside the others' effective coverage would be the way to exhibit their effect directly, and is a natural item for further testing.

8. Choosing the Local Planner

The navigation chapter described the division of labour between the global planner, which lays out a route across the whole map, and the local controller, which follows that route while avoiding obstacles in real time. The behaviour of the system at runtime — how quickly it reaches its goal, how much clearance it keeps, and how it copes when obstacles move — is determined largely by which local controller is used. The navigation framework offers several, embodying genuinely different strategies. To make an informed choice for the mobile manipulator rather than accepting a default, a dedicated comparative study of the candidate local planners was carried out.

Because the question being studied — how each planner balances speed, efficiency, and safety in static and moving-obstacle conditions — concerns the planner's own behaviour and not the specific robot carrying it, the comparison was run on a simpler differential-drive platform in a controlled test world. This keeps the experiment tractable and repeatable: a small, well-characterised robot in a purpose-built corridor isolates the planners' differences far more cleanly than the full mobile manipulator would, while the conclusions about which planner suits which conditions transfer directly back to the project robot, whose base is likewise differential-drive.

8.1 Aim of the Study

Three local planners shipped with the navigation stack were compared: the dynamic-window approach (DWB), a sampling-based predictive controller (MPPI), and a regulated pure-pursuit controller (RPP). Each was evaluated under two conditions: a static environment with fixed obstacles, and a dynamic environment in which several obstacles follow repeatable moving trajectories. The study set out to answer four practical questions — which planner reaches the goal fastest, which produces the most efficient path, which keeps the largest safety margin, and how each degrades when obstacles begin to move. Only the planner and the obstacle condition were varied; everything else was held constant so that any difference is attributable to the planner alone.

8.2 Experimental Setup

A custom corridor world was authored for the study: an L-shaped space roughly thirteen metres by nine, bounded by walls and containing four fixed cylindrical obstacles along its length. An occupancy map of the corridor was built by driving the robot through it under the mapping system and saving the result, in exactly the manner described for the project's own test environment. In the dynamic condition, a dedicated component drives a set of cylindrical obstacles along fixed, repeatable paths, so that every planner faces an identical sequence of moving obstacles and the comparison remains fair.

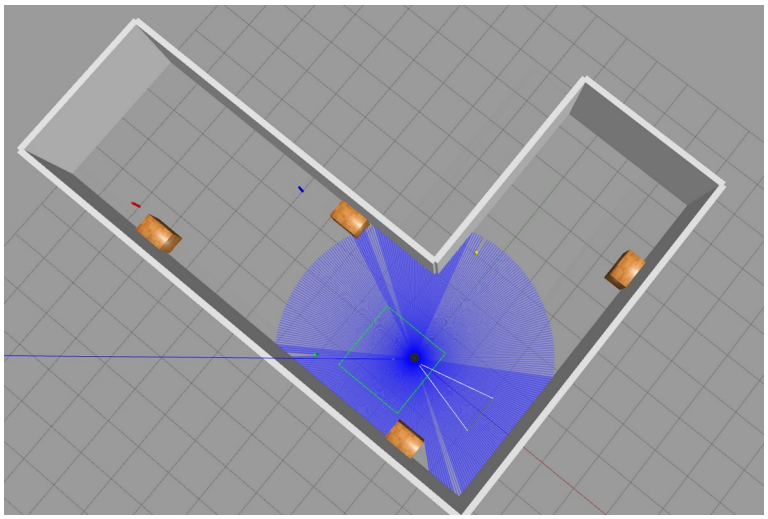


Figure 8.1 — *The L-shaped corridor used for the planner study. The differential-drive test robot sits at the centre with its laser scan shown; four cylindrical obstacles are placed along the corridor.*

Three controller configurations were prepared, one per planner, each left at its documented default parameters so that the comparison reflects out-of-the-box behaviour rather than hand-tuned optimisation. The global costmap, the local costmap, and all other settings were identical across the three, so the only thing differing between trials was the local planner itself.

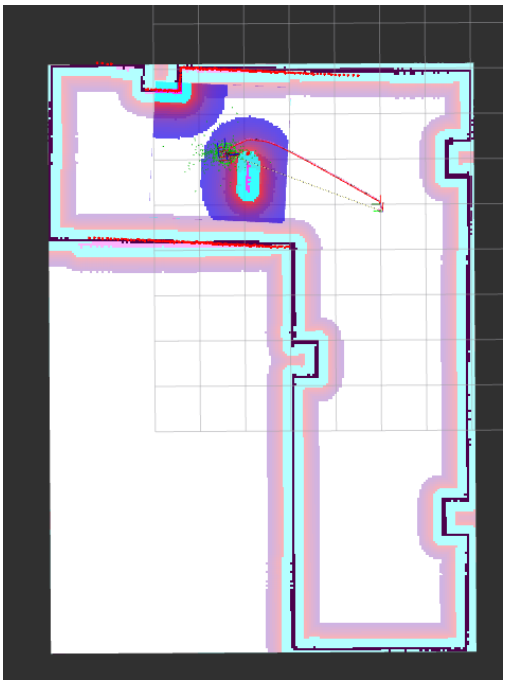


Figure 8.2 — *A trial in progress: the static map overlaid with the global costmap inflation, with the robot navigating around a moving obstacle toward the goal.*

8.3 Procedure and Metrics

The benchmark comprised six cells — three planners under two conditions — with twenty trials per cell, for one hundred and twenty trials in total. The same twenty start-and-goal pairs were reused across every cell, which allows the planners to be compared on identical tasks. The pairs

were drawn from across the free space of the corridor to cover both straight runs and around-the-corner navigation, and were filtered to avoid positions too close to walls or obstacles.

Each trial followed the same fixed protocol without manual intervention: obstacles were parked and the robot teleported to the start, localisation was re-seeded and the costmaps cleared and allowed to repopulate, then the obstacles were released and the goal dispatched, after which the robot's pose, speed, and distance to obstacles were sampled until it either reached the goal or hit a time limit. To remove localisation noise as a confounding factor, the ground-truth pose from the simulator was used for logging, so the comparison reflects the planners alone. The per-trial metrics are summarised below.

Metric	Meaning
success	Goal reached within tolerance, without timing out
time to goal	Seconds from goal dispatch to completion
path length	Total distance travelled along the trajectory
path efficiency	Straight-line distance divided by path length
min clearance	Closest the robot came to any obstacle
mean jerk	Smoothness of motion (lower is smoother)
close approaches	Number of samples spent very close to an obstacle

8.4 Results

Across the static condition all three planners succeeded on every trial and behaved broadly similarly; the meaningful differences emerged under moving obstacles. The summary figures below report the picture across all twenty trials per cell.

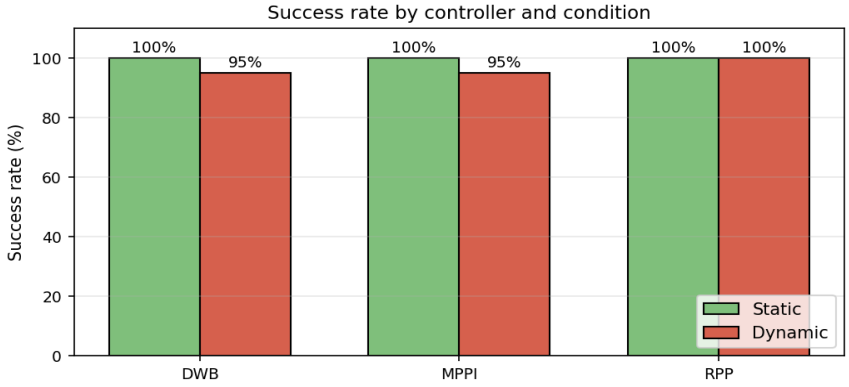


Figure 8.3 — *Success rate by planner and condition. All three succeed on every static trial; under dynamic conditions the dynamic-window and predictive planners each miss one trial, while pure-pursuit completes all twenty.*

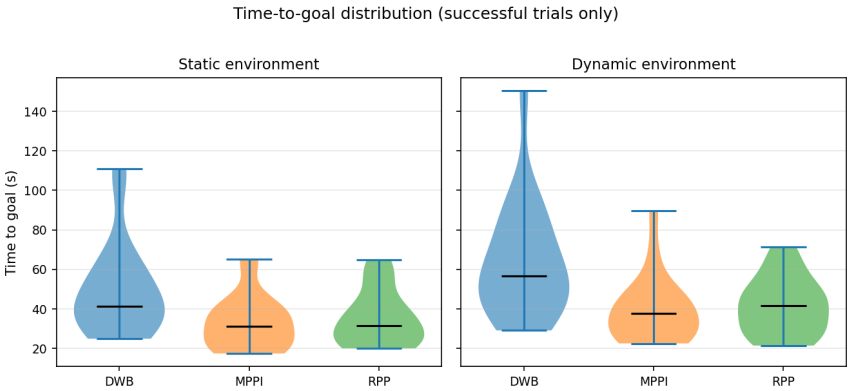


Figure 8.4 — *Time-to-goal distributions (successful trials only). The dynamic-window planner is consistently the slowest in both conditions; the predictive and pure-pursuit planners are similarly faster.*

On time-to-goal the dynamic-window planner was consistently the slowest, while the other two were markedly quicker and statistically indistinguishable from one another. On path efficiency the pure-pursuit planner produced the straightest routes, with the predictive planner close behind and the dynamic-window planner taking the most circuitous paths owing to its sampling-based exploration.

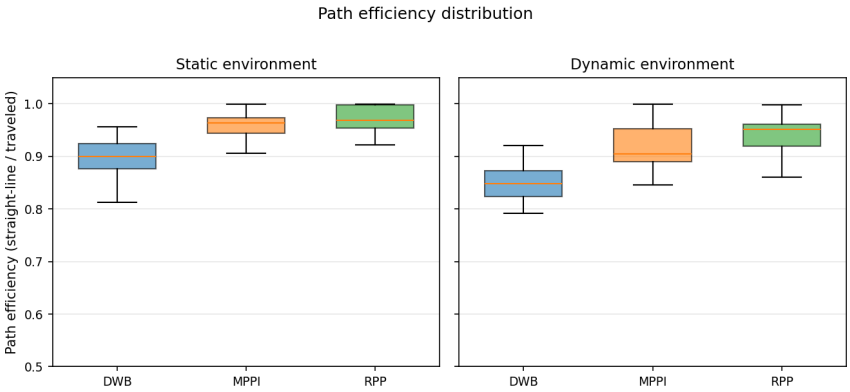


Figure 8.5 — *Path efficiency. The pure-pursuit planner is most efficient, the predictive planner close behind, and the dynamic-window planner least efficient.*

The safety picture is where the planners diverge most sharply, and it inverts the speed ranking. The dynamic-window planner kept the largest clearance and recorded the fewest close approaches; the pure-pursuit planner, fastest and most efficient, kept the smallest margin and recorded by far the most close approaches under moving obstacles; the predictive planner sat between the two. The planners trade speed against safety, and no single one is best on every axis.

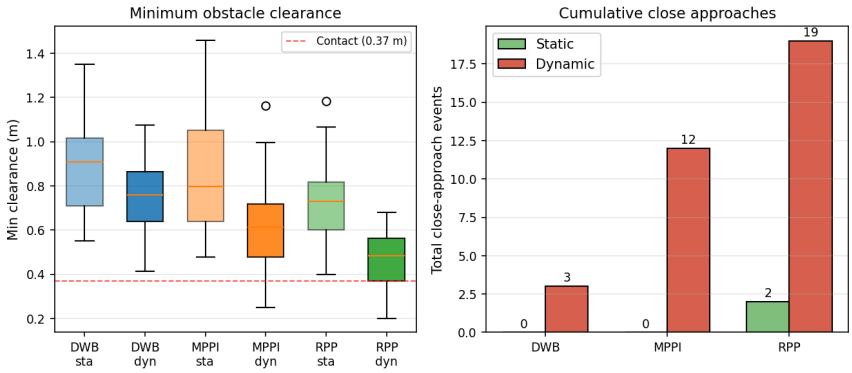


Figure 8.6 — *Minimum clearance (left) and cumulative close approaches (right). The dynamic-window planner is safest; the pure-pursuit planner accumulates the most close approaches under dynamic conditions — a clear speed-versus-safety trade-off.*

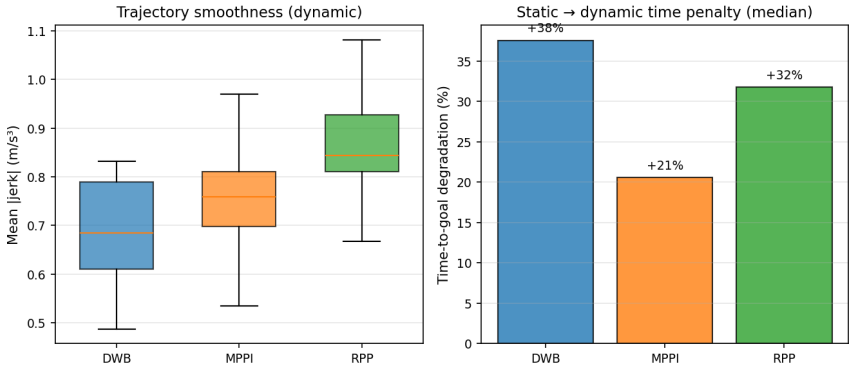


Figure 8.7 — *Motion smoothness (left) and the time penalty incurred moving from static to dynamic conditions (right). The predictive planner degrades least when obstacles start moving, thanks to its anticipatory horizon.*

8.5 Discussion and Choice

The three planners occupy distinct positions on the trade-off between speed and safety, and the right choice depends on the environment. The dynamic-window planner is the conservative option: the largest clearance and fewest close approaches, but the slowest, with the heaviest time penalty once obstacles move. The pure-pursuit planner is the geometric option: the most efficient and among the fastest, but with the smallest safety margin and by far the most close approaches under moving obstacles, which would be a concern in a space shared with people. The predictive planner is the balanced option: nearly as fast and efficient as pure-pursuit, but with a noticeably larger clearance and fewer close approaches, and it degrades the least when obstacles begin to move because its predictive horizon anticipates them.

For the mobile manipulator, which is intended to operate in environments that may contain moving people and objects, the balance of speed and safety matters more than raw speed alone. The predictive planner's combination of competitive speed with a healthier safety margin and the gentlest degradation under moving obstacles makes it the most suitable default for this robot, with the conservative dynamic-window planner a reasonable fallback where maximum clearance is the priority.

A caveat applies, as with any such comparison: all three planners were run at their default parameters, and per-planner tuning could shift the ranking. The single corridor exercises corner navigation and parallel obstacle motion but may not represent every environment, and the moving obstacles followed deterministic paths rather than the more variable motion of real people. The

study is therefore best read as a principled basis for the initial choice of planner, not as the final word on tuning.